ASDXDDL8M3 Data Modelling and Management Day 1 Summary

Monday, October 19th, 2026

Overview

This session, led by QA Trainer Ninad Rampure, continues hands-on instruction in data modeling, normalization, and schema design with a focus on a movie-database group activity and detailed data-type selection. Key participants who contributed include Ryan, Charlie Maslen, Andrew McFarlane, Josh Rankine, Tracy, Chris Cassar, Jonathan Kermath, and several others who reported lab access issues.

Main topics and outcomes

- Movie activity and deliverables: Ninad assigns groups to deliver five entities (three primary and two bridges), a diagram, and a completed data dictionary; he instructs teams to mark primary keys and nullability and to complete the Word template during breakout work.

- Deep dive on data types: Ninad explains numeric types in detail—DECIMAL precision (P) and scale (S), and when to use FLOAT/REAL for approximate values—and covers DATE/TIME choices for date-only versus timestamp needs; he urges deliberate selection of P/S for monetary and numeric columns.

- Character and Unicode guidance: Ninad recommends NVARCHAR for multilingual flexibility, explains CHAR vs VARCHAR/NVARCHAR and MAX usage for long free-text fields, and warns about the doubled byte cost of Unicode which affects max character counts.

- Practical advice and tooling: Ninad reiterates avoiding deprecated types (TEXT/NTEXT), points learners to a reference table in the delivery notes, and asks groups to focus on core types for this exercise.

- Breakout logistics and follow-up: The trainer splits attendees into four breakout groups, sets the work period until 16:30, offers screen-sharing permissions, and will follow up tomorrow to finish outstanding items and enable the shared ERD portal when permissions are resolved.

Decisions and commitments

- Groups will populate the Word data dictionary with entity names, attributes, data types, primary keys, and nullability during the breakout session and may complete diagrams now or tomorrow (trainer instruction).

- Ninad will assist with breakout membership and screen-sharing permissions and will continue unfinished activity the next morning (trainer action).

Concerns and notes

- Several participants reported launcher or portal blocks earlier; Ninad will follow up with cloud PC provisioning and training keys as needed.

- Participants should re-read the decimal P/S guidance and consult the delivery-note reference when choosing numeric precisions and character lengths.

Overall, the meeting balances conceptual modeling tasks with detailed, actionable guidance on data types and breakout execution: participants receive specific rules for decimals, floats, date/time, and character types and then proceed into breakout groups to apply those choices to the movie-database data dictionary and diagrams.

Detailed Summary

Electricity bill data

The trainer walks through the data elements present on an electricity bill and how those elements map to separate database tables. He explains why account numbers, meter numbers, tariff, meter readings, standing charges, and previous bills are stored separately and combined at invoice generation time. The current state is conceptual: data is stored across multiple tables and invoice totals are computed from those sources rather than stored as persistent records.

- QA Trainer Ninad Rampure says account numbers link customers to meters and tariffs and determine unit pricing.
- QA Trainer Ninad Rampure says meter readings, tariff, and standing charges combine during processing to produce an invoice outcome that is not itself stored in the database.
- QA Trainer Ninad Rampure says customer details (name, address, phone) are stored as customer data in a dedicated table.

Movie database example

The trainer uses a movie catalog to illustrate entities and attributes: title, director, actors, roles, release date, genre, revenue, duration, awards, and rankings. He emphasizes the need for structured storage (actor and director tables, role/casting relationships) so queries like 'movies with Johnny Depp' or 'movies by director X' return correct results. The state is a conceptual model showing how unique identifiers and properly structured columns enable accurate searching and filtering.

- QA Trainer Ninad Rampure explains titles, directors, actors, roles, release date, genre, revenue, and duration are data attributes that help find movies.
- Ryan notes that users search by film name, category, run time, actors, directors, and sales data.
- QA Trainer Ninad Rampure says missing structured actor data prevents finding films where an actor had a cameo.
- QA Trainer Ninad Rampure says the movie activity will be deployed tomorrow and that groups must identify entities, bridges, and data types today.

Retail bill data

The trainer compares supermarket bills to other datasets, highlighting product items, unit prices, store location, transaction date, payment method, customer loyalty IDs,

and quantities as critical data points. He explains how loyalty cards link purchases to customers so businesses analyze shopping patterns and tailor offers seasonally. Currently, these data points exist across different systems and must be modeled to support business processes like promotions, stock planning, and trend analysis.

- Chris Cassar says retail bills include item, price, date, and store location.
- QA Trainer Ninad Rampure says loyalty card IDs link transactions to customer accounts and enable behavioral analysis.
- QA Trainer Ninad Rampure says retailers use product, price, seasonal trends, and loyalty data to craft offers and manage stock.

Purpose of data modeling

The trainer describes modeling as the process of collecting enterprise information, identifying entities (tables), and building relationships so different applications and business functions can use consistent data. Modeling eliminates redundancy, enforces single-source-of-truth, supports planning and business processes, and prepares data for relational databases. The state is an educational overview: students will learn entity identification, normalization, logical modeling, and implementation in upcoming lessons.

- QA Trainer Ninad Rampure says data modeling organizes data across multiple tables to support business functions and reduce redundancy.
- QA Trainer Ninad Rampure says modeling enables performance planning, stock management, product diversification, and seasonal offers.
- QA Trainer Ninad Rampure says modeling leads into normalization and logical data modeling before loading into a relational database.

Airline entity relationships

Using airlines as an example, the trainer clarifies how passengers, flights, and aircraft relate: a flight at a given time has one assigned aircraft and many passengers, and passengers can take many flights. He advises avoiding redundant many-to-many mappings by modeling flights (instances) that link passenger bookings and aircraft assignments. The state is conceptual guidance for reducing redundancy and enabling straightforward queries like passenger lists per flight.

- QA Trainer Ninad Rampure says one flight instance links to a single aircraft and to many passengers checked-in for that flight.
- QA Trainer Ninad Rampure says modeling flights as the linking entity avoids redundant many-to-many mappings between passengers and aircraft.
- QA Trainer Ninad Rampure says querying the flight record returns both aircraft and passenger information for that route/time.

Entities and attributes definition

The trainer defines entities as nouns (person, place, object, event) that have attributes and require a primary key. Attributes describe entities and must be atomic (single-valued) to enable reliable querying. He stresses planning which attributes belong to which entity and separating composite values (like address) into multiple columns. The

state is instructional: participants learn to identify keys and attributes and to decide required versus optional fields.

- QA Trainer Ninad Rampure says each entity needs a unique primary key and attributes must depend on that key.
- QA Trainer Ninad Rampure says attributes must hold a single value and composite attributes (e.g., address) should be split into separate columns.
- QA Trainer Ninad Rampure says attributes that logically belong to other entities (e.g., department) become foreign keys.

Keys: primary and surrogate

The trainer explains key selection: keys should be simple and unique; names or titles are typically poor keys due to duplicates. He introduces surrogate keys for entities lacking natural unique identifiers and composite/superkeys when multiple attributes together ensure uniqueness. The current state: learners should choose appropriate keys (create surrogate keys where needed) for each entity.

- QA Trainer Ninad Rampure says keys should be simple and unique and that names or titles often repeat and are therefore unsuitable keys.
- QA Trainer Ninad Rampure says surrogate keys (generated IDs) are appropriate when no natural unique identifier exists for an entity.
- QA Trainer Ninad Rampure says composite and super keys combine attributes when needed to ensure uniqueness.

Order processing example

The trainer walks through an order model: orders have order numbers, dates, values; customers have customer numbers and contact details; order items link orders to products and quantities; employees (sales IDs) may be optional attributes if a salesperson handled the order. He clarifies that quantities and processing calculations are transactional/process data, whereas product and customer details are entity data. The state is a conceptual mapping showing how order, customer, product, and employee entities join to form invoices.

- QA Trainer Ninad Rampure says an order entity should include order number as primary key, date, and order value.
- QA Trainer Ninad Rampure says order items link an order to product IDs and record quantities used for processing, not as static entity attributes.
- QA Trainer Ninad Rampure says salesperson (employee ID) can be an optional attribute and should link to an employee entity when present.

Normalization and data formats

The trainer defines normalization as standardizing, validating, and enforcing consistent formats before persisting data to avoid inconsistencies (e.g., date formats or missing addresses). He outlines a flow: collect external data, analyze and model entities and relationships, normalize data types and formats, then implement logical models in a relational database. The state is an instructional checklist for preparing clean, queryable data.

- QA Trainer Ninad Rampure says normalization fixes inconsistencies like varying date formats and missing data before database insertion.
- QA Trainer Ninad Rampure says the process moves from data collection to entity modeling to logical modeling to relational database implementation.
- QA Trainer Ninad Rampure says consistent data types and formats are required to enable accurate queries and processing.

First/second/third normal forms

The trainer explains the first three normal forms in sequence: first normal form requires atomic columns and consistent data types; second normal form requires that non-key attributes depend on the whole primary key and removes partial dependencies into new tables; third normal form requires that non-key attributes depend only on the primary key and removes transitive dependencies by using foreign keys. The state is instructional with a practical demo workbook showing progressive transformation from unnormalized data to third normal form.

- QA Trainer Ninad Rampure says first normal form requires unique columns with atomic values and consistent data types to avoid repeating groups.
- QA Trainer Ninad Rampure says second normal form requires identifying entities and ensuring attributes fully depend on the whole primary key so partial dependencies move to new tables.
- QA Trainer Ninad Rampure says third normal form requires that non-key attributes depend only on the key and that transitive dependencies be separated into related tables.

Normalization anomalies explained

The trainer revisits update, insertion, and deletion anomalies and uses the example workbook to show how moving from first to third normal form prevents inconsistency in product pricing, avoids insert blocks when parent records are missing, and prevents accidental data loss when deleting rows. The state is heightened understanding with live examples pointing to duplicated product prices and misplaced totals that must be corrected.

- QA Trainer Ninad Rampure demonstrates an update anomaly where the same product appears with different prices and explains why a single canonical price is required.
- QA Trainer Ninad Rampure explains insertion anomalies by showing cases where dependent records cannot be created without parent entities.
- QA Trainer Ninad Rampure says deletion anomalies can remove needed information and must be prevented by separating entities properly.

Practical normalization exercise

Participants work through a supplied unnormalized sales table and, guided by the trainer, convert it to first, second, and third normal forms by splitting into product, order, customer, publisher, and order_items (bridge) tables. The activity focuses on eliminating repeated columns, establishing primary keys, and placing quantity and

price in the bridge table to preserve transactional context. The state is that most participants completed the exercise and validated the bridge-table solution with the trainer.

- QA Trainer Ninad Rampure instructs learners to open the provided unnormalized and normalization example files and to follow the stepwise conversion to third normal form.
- Ryan describes his solution: product table with product_id, orders table with order_number and date, and an order_details bridge table containing order_number, product_id, and quantity.
- QA Trainer Ninad Rampure confirms the bridge (order_items) is the correct place for quantity and item-level price/discount reasons.

Many-to-many bridge tables

The trainer clarifies that many-to-many relationships (e.g., orders–products, actors–movies) require a bridge table to store relationship-specific attributes like quantity or role name. He demonstrates the bridge table's operational use (e.g., warehouse picking, invoice generation) and suggests including columns for discounts or item condition when values differ per relationship row. The state is prescriptive: learners should create bridges for many-to-many links and include transactional attributes there.

- QA Trainer Ninad Rampure says many-to-many relationships require a bridge table that can hold relationship-specific attributes such as quantity, discount, or condition.
- QA Trainer Ninad Rampure says warehouse staff and invoicing processes typically operate from the order_items bridge table to determine pick lists and invoice values.
- QA Trainer Ninad Rampure suggests adding columns in the bridge to explain differing item prices (e.g., discount, refurb) when they occur.

Practical lab setup issues

The trainer coordinates lab access and resolves practical connectivity and launcher installation problems for students using two lab delivery methods: 'Go to my PC' and QA Learn On Demand (LOD). Several participants report organizational blocks to the portal or to installing the launcher; the trainer collects names, shares credentials in chat, and provides guidance on shortcuts and next steps. The state is that most users are connected, some remain blocked, and the trainer will request cloud PCs or training keys and follow up.

- QA Trainer Ninad Rampure instructs students to check direct chat for lab username, password, and access code and to copy them exactly.
- Andrew McFarlane, Robyn Brock, Ayotunde Akintunde, Tracy, and Rachel Davies report the portal or launcher is blocked by their organizations.
- QA Trainer Ninad Rampure says he will request cloud PCs for blocked users and will provide LOD training keys for those on the QA Learn On Demand platform.

Lab onboarding and shortcuts

The trainer shows go-to-my-PC users how to create a desktop shortcut for the lab launcher to simplify future logins and explains where to find required applications on the remote desktop. He also outlines the sign-in flow for LOD users (create account, input training key) and confirms which participants have successful access. The state is actionable setup guidance and follow-up tasks to ensure everyone can join tomorrow's labs.

- QA Trainer Ninad Rampure advises go-to-my-PC users to create a launcher shortcut on the remote desktop to save time for the next session.
- QA Trainer Ninad Rampure instructs LOD users to create or sign in to a QA Learn On Demand account so he can supply the training key.
- Josh Rankine and Tracy confirm they successfully accessed the lab portal during the session.

Activity: many-to-many challenge

The trainer issues an in-class challenge to break a many-to-many order-product problem into appropriate tables and asks participants to produce two or three primary tables plus a bridge that carries quantity. Groups convert an orders/products/customer unnormalized table into order, product, customer, and order_items tables to satisfy third normal form. The state is interactive: most participants completed the task and compared solutions before the break.

- QA Trainer Ninad Rampure asks learners to split an unnormalized sales table into appropriate entities and to create a bridge table for quantities.
- QA Trainer Ninad Rampure instructs participants to preserve all original attributes while removing repetition and ensuring keys are unique.
- Participants generally complete the exercise and the trainer provides a model solution in the delivery notes for later review.

Data types and schema design

Before group work, the trainer explains choosing column data types (e.g., integer for IDs, varchar/nvarchar for text, currency for budgets) and the importance of specifying field lengths and appropriate types to prevent data-entry and export issues. He recommends nvarchar for character data to support wide character sets and suggests reasonable length limits (e.g., 50 chars for names). The state is preparatory: participants will apply these rules when building data dictionaries for the movie activity.

- QA Trainer Ninad Rampure says ID columns should use integer types (INT) and that text fields should use character types such as NVARCHAR with a defined length.
- QA Trainer Ninad Rampure recommends using a currency or decimal data type for monetary fields and setting sensible precision/scale.
- QA Trainer Ninad Rampure suggests setting a character length (e.g., 50 for names) to catch unexpected data and enforce constraints.

Movie activity assignment

The trainer launches the movie-scenario group activity: groups must identify five entities (three primary and two bridge tables), propose a diagram, and compile a data dictionary with attributes, data types, keys, and constraints. He points learners to the Excel movies/actors tab and a Word activity file and instructs teams to produce entity/relationship plans for deployment tomorrow. The state is collaborative: groups prepare diagrams first, then fill attribute tables with types and constraints.

- QA Trainer Ninad Rampure assigns a group exercise to design a movie database and asks groups to identify five entities including actor–movie bridges.
- QA Trainer Ninad Rampure instructs groups to produce a diagram and a data dictionary for each entity including keys, data types, and mandatory/optional constraints.
- QA Trainer Ninad Rampure advises teams to draw diagrams first to clarify relationships and then populate attribute data types and constraints.

Movie entities identification

During group planning the class identifies three primary movie-domain entities—movies, directors, actors—and two bridge entities to resolve many-to-many relationships (e.g., movie_actors and movie_directors or roles). The trainer confirms these choices and instructs groups to propose diagrams that show how transitive dependencies and bridges connect the core entities. The state is set: groups will meet in breakout sessions to finalize diagrams and data dictionaries.

- Charlie Maslen and others identify Movies, Directors, and Actors as primary entities for the movie database.
- Participants and QA Trainer Ninad Rampure agree on two bridge entities (e.g., movie_actors and movie_directors or character-role tables) to break many-to-many links.
- QA Trainer Ninad Rampure instructs groups to propose entity diagrams and then produce data dictionaries specifying keys and data types.

Attribute constraints and nulls

The trainer instructs participants to record attribute constraints (mandatory vs optional) for each entity and to explicitly mark primary keys and foreign keys in their data dictionaries. He highlights that director and other core tables should have NOT NULL constraints on identifying fields and that planning which attributes allow NULL affects data integrity. The state is an actionable checklist: groups must fill the Word template with entity names, attributes, data types, primary keys, and nullability before breakout work.

- QA Trainer Ninad Rampure says the third column in the data dictionary should specify whether an attribute is mandatory (NOT NULL) or optional.
- QA Trainer Ninad Rampure instructs groups to list primary keys for the first three primary tables and optionally mark foreign keys with an asterisk or name them explicitly.
- Andrew McFarlane and QA Trainer Ninad Rampure confirm director name and other identifying fields should be NOT NULL in the director table.

ER diagram tooling

The trainer introduces diagramming tools and shares links (Microsoft Transact-SQL data types page, learning sites, and Visual Paradigm) so teams can draw ER diagrams online. He demonstrates how to add entities, attributes, and relationships (one-to-one, one-to-many) and encourages one group member to maintain a shared diagram for collaboration. The state is exploratory: the trainer will let groups plan in Word for now but intends to migrate work onto the shared diagram portal when ready.

- QA Trainer Ninad Rampure shares a Visual Paradigm ERD tool and demonstrates adding entities, attributes, and relationship arrows to build an ER diagram.
- QA Trainer Ninad Rampure says one person in each group can prepare the diagram on the portal and then share it with teammates for collaboration.
- QA Trainer Ninad Rampure will let groups use the Word planning document first and will enable the shared portal later once permissions are fixed.

Detailed data types guidance

The trainer reviews SQL data type families (integers, bits, decimals, money, characters, unicode, binary, date/time) and explains selection criteria for size and storage implications. He emphasizes choosing appropriate integer variants (tinyint, smallint, int, bigint) based on expected value ranges, using bit for boolean-like fields, and using money/decimal for currency columns. The state is educational: participants should map sample Excel values to specific Transact-SQL types when populating their data dictionaries.

- QA Trainer Ninad Rampure explains integer variants (tinyint, smallint, int, bigint) and advises selecting the smallest type that safely covers expected values.
- QA Trainer Ninad Rampure says use BIT for boolean-like fields (true/false) and MONEY or DECIMAL for budget and revenue fields.
- QA Trainer Ninad Rampure notes each data type has a storage size in bytes and that cumulative storage matters only at very large scales but still informs correct type choice.

Storage size and practical selection

The trainer places data type selection in practical context by describing bit/byte/kilobyte growth and explaining why storage-size differences among SQL types are generally negligible for classroom exercises. He nonetheless advises logical type selection to avoid inappropriate values and to maintain consistency with existing Excel data; for example, money types should be used where dataset values clearly represent currency. The state is guidance-focused: choose correct types primarily for correctness, not storage optimization in this exercise.

- QA Trainer Ninad Rampure explains that bytes accumulate into kilobytes, megabytes, and gigabytes, but the per-column byte differences are minor for typical datasets.

- QA Trainer Ninad Rampure advises using standard types (e.g., INT, MONEY) that match existing Excel values rather than over-optimizing for storage.
- QA Trainer Ninad Rampure emphasizes that correct logical type choice prevents invalid data and supports downstream processing even if storage impact is small.

Decimal precision/scale

The trainer explains decimal's precision (P) and scale (S): precision is the total number of digits allowed (left and right of the decimal), while scale is the number of digits to the right of the decimal. He demonstrates how P and S determine allowable digits on each side and recommends default/typical values while cautioning that picking P/S affects storage size and precision needs. The state is instructional: learners should choose P and S deliberately for monetary and numeric columns and review the guidance when completing the activity.

- QA Trainer Ninad Rampure explains that precision (P) defines the total maximum digits and that scale (S) defines digits to the right of the decimal.
- QA Trainer Ninad Rampure gives examples showing how P and S (e.g., P=6, S=3) allocate digits to left and right of the decimal point and influence allowable values.
- QA Trainer Ninad Rampure advises learners to read the decimal P/S guidance again when deciding numeric data types for their activity.

Float and real selection

The trainer describes float and real as approximate numeric types used when decimal precision varies widely and a fixed P/S cannot capture the column's variability. He explains float's default precision choices (float(53) approximates ~15 digits) and how lower float values reduce supported digits, and advises using float for highly variable decimal data where exact precision is not required. The state is advisory: choose float/real only when variability prevents a meaningful decimal specification and accept approximation behavior.

- QA Trainer Ninad Rampure says float and real provide approximate values and are appropriate when decimal digits vary too much to pick a single P/S.
- QA Trainer Ninad Rampure explains float defaults (e.g., float(53)) and maps those values to approximate digit coverage when deciding which float precision to use.
- QA Trainer Ninad Rampure recommends float when a column contains highly variable decimal lengths that cannot be normalized into a single decimal P/S.

Date/time type guidance

The trainer outlines choices among date and time types: use date when only day/month/year matters and date/time2, datetimeoffset, or small datetime when time-of-day or timezone-aware timestamps are required. He emphasizes selecting the narrowest type that preserves needed granularity (e.g., date for release dates,

datetime2 for precise order timestamps). The state is directive: pick date-only types for release dates and richer datetime types for transactional timestamps.

- QA Trainer Ninad Rampure says use DATE for attributes like release date when time is not needed.
- QA Trainer Ninad Rampure recommends DATE/TIME2 or DATETIMEOFFSET when you need hours, minutes, seconds, or timezone context for transactions.
- QA Trainer Ninad Rampure emphasizes choosing the right datetime type to preserve required granularity for auditing and transactional purposes.

Character and Unicode types

The trainer explains fixed (CHAR) versus variable (VARCHAR/NVARCHAR) character types and the difference between non-Unicode and Unicode storage, advising NVARCHAR for multilingual data. He clarifies MAX usage for long free-text fields (VARCHAR(MAX) / NVARCHAR(MAX)) and how Unicode doubles bytes per character, affecting max character counts—e.g., NVARCHAR(MAX) supports ~4000 characters versus VARCHAR(MAX) ~8000. The state is prescriptive: use NVARCHAR for flexibility and multilingual support, CHAR for strictly fixed-length values, and MAX only when length is unknown or very large.

- QA Trainer Ninad Rampure explains CHAR is fixed-length and NVARCHAR/VARCHAR are variable-length and therefore more space-efficient for variable text.
- QA Trainer Ninad Rampure says NVARCHAR supports Unicode and is required for multilingual data such as Chinese or Arabic names.
- QA Trainer Ninad Rampure recommends NVARCHAR(MAX) for long free-text fields and notes NVARCHAR(MAX) stores fewer characters than VARCHAR(MAX) because Unicode uses two bytes per character.

Collation and advanced types note

The trainer briefly mentions advanced types (binary, geography, JSON, geometry, XML, vector) and deprecations (TEXT/NTEXT) and advises sticking to core types for this exercise to avoid confusion. He warns that some legacy types are being removed in newer SQL versions and suggests consulting the provided document for specific guidance. The state is advisory: focus on core numeric, character, and datetime types for the activity and refer to delivery notes for additional types.

- QA Trainer Ninad Rampure warns that legacy types like TEXT and NTEXT are deprecated and should be avoided.
- QA Trainer Ninad Rampure notes there are specialized types (geography, JSON, XML, vector) but recommends students focus on core types for this class activity.
- QA Trainer Ninad Rampure says the delivery notes include a reference table to help pick appropriate data types and lengths.

Breakout logistics and timing

The trainer divides students into four breakout groups, sets the session duration (until 16:30), and instructs groups to complete their Word data dictionary and, if time permits, a diagram. He offers to grant screen-sharing rights, to reassign participants where needed, and to continue unfinished work the following morning. The state is operational: breakout sessions start immediately and the trainer will monitor group membership and participation.

- QA Trainer Ninad Rampure assigns four breakout groups and tells participants the breakout work runs until 4:30 PM.
- QA Trainer Ninad Rampure says one group member can share their screen for diagram work and he will enable screen-sharing permissions on request.
- QA Trainer Ninad Rampure commits to continuing any unfinished activity tomorrow morning and to reorganize groups if participants reconnect.

Breakout join troubleshooting

During breakout initiation a participant reports not receiving the join request; the trainer acknowledges and reattaches the participant to resolve access. This highlights minor connectivity or Webex request issues during grouping. The state is resolved in-session with trainer assistance so participants can join their assigned breakout rooms.

- Jonathan Kermath reports he has not received a breakout request and QA Trainer Ninad Rampure reattaches him to the session.
- QA Trainer Ninad Rampure monitors breakout joins and adjusts group membership (e.g., Josh) to balance groups and accommodate rejoining participants.